

OpenCourseWare
**Procesamiento de Lenguaje Natural con
Aprendizaje Profundo,**
Máster en Ciencia y Tecnología Informática

**Tema 3: Redes recurrentes para Procesamiento
de Lenguaje Natural**

Objetivos

- Conocer los fundamentos de las redes recurrentes (en inglés, recurrent neural networks (RNN)).
- Conocer los tipos de redes recurrentes y las características de sus neuronas.
- Aprender a usar redes recurrentes para resolver tareas de PLN.

Índice

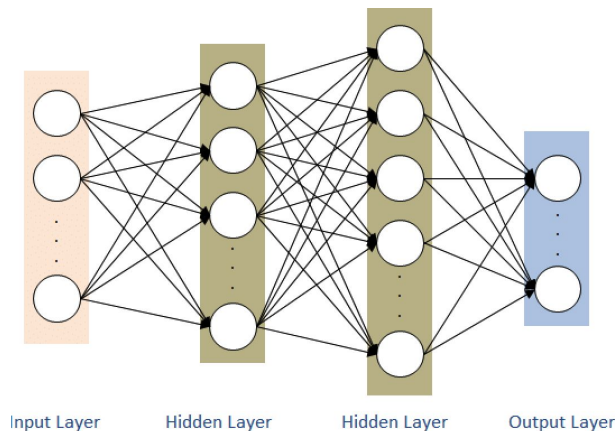
- Redes recurrentes
- Tipos de RNN
 - básica (vanilla)
 - LSTM
 - GRU
- Aplicaciones de RNN en PLN

Limitaciones de las feed-forward para PLN

Una red feed-forward *típica* **trata igual** a estas dos oraciones: “El gobierno aprobó un decreto” y “El aprobó un gobierno decreto”

porque:

- cada entrada (token) se procesa de forma aislada
- no tiene memoria
- no modelan secuencias



¿Por qué el lenguaje natural es difícil para las redes neuronales básicas?

- El lenguaje humano es secuencial
- El significado depende del contexto
- El orden importa

Ejemplo

*Juan es coruñés. Se crió en un barrio muy humilde y, durante sus primeros años, trabajó como chico de los recados antes de emigrar a Madrid. **Él habla un perfecto ?***

Para predecir o interpretar una palabra:

- no basta con mirarla de forma aislada,
- necesitamos **recordar información previa**, a veces lejana en la secuencia.

Redes recurrentes (RNN)

- Una **red recurrente** es una red neuronal pensada para **procesar secuencias** de datos

$$\mathbf{x} = (\mathbf{x}_{(1)}, \mathbf{x}_{(2)}, \dots, \mathbf{x}_{(n)}).$$

Redes recurrentes (RNN)

- Dada una secuencia de palabras, el modelo debe **predecir la siguiente palabra**:

El *gobierno* *aprobó* *un* **?**

$\mathbf{x}_{(1)}$ $\mathbf{x}_{(2)}$ $\mathbf{x}_{(3)}$ $\mathbf{x}_{(4)}$

decreto

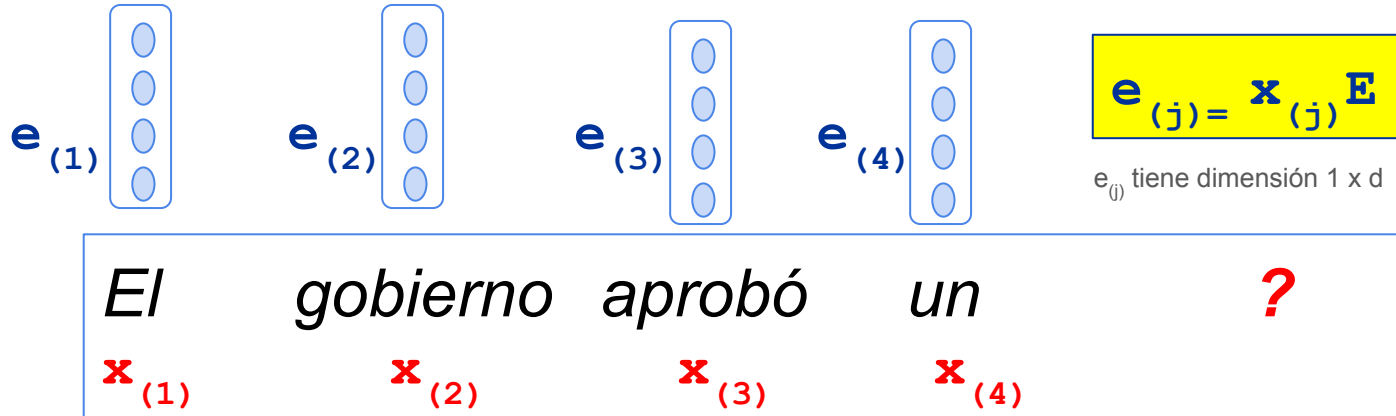
plan

acuerdo

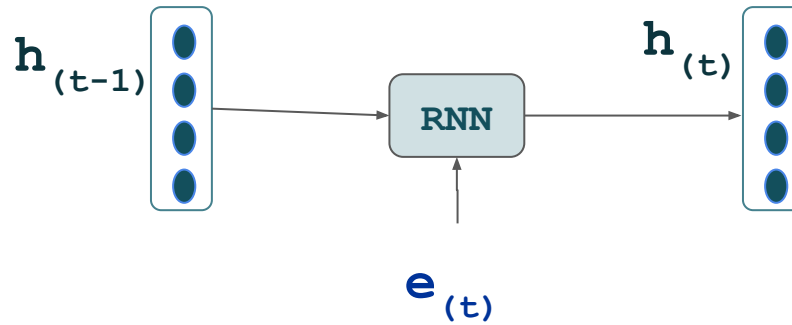
aumento

Capa de entrada

- Cada paso temporal (token) se representa con un **embedding**.
- La capa de entrada está formada por los embeddings de la secuencia.

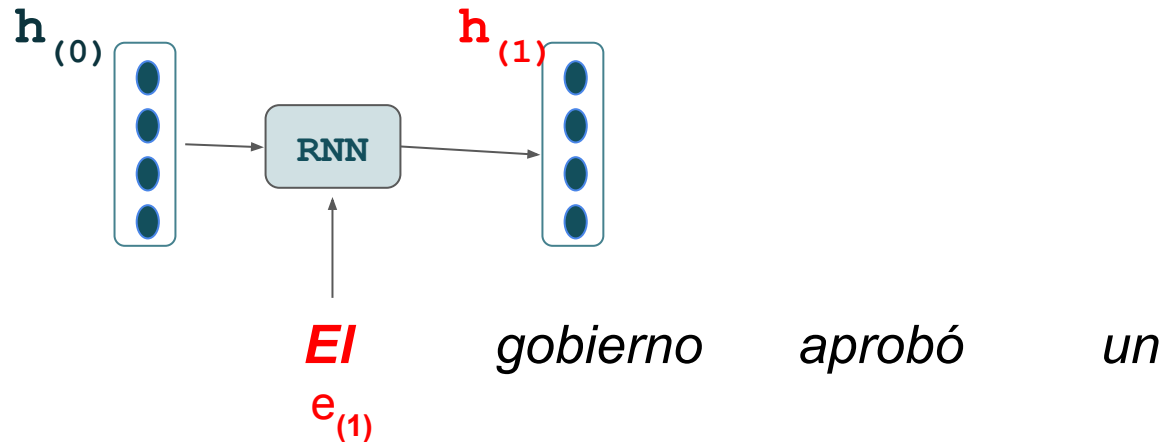


Redes recurrentes (RNN)

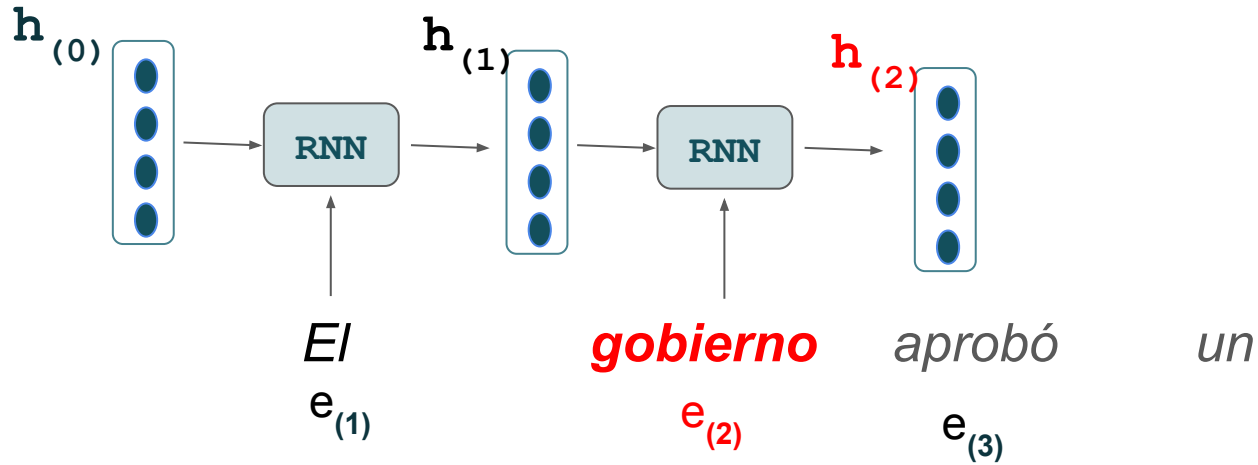


Redes recurrentes (RNN)

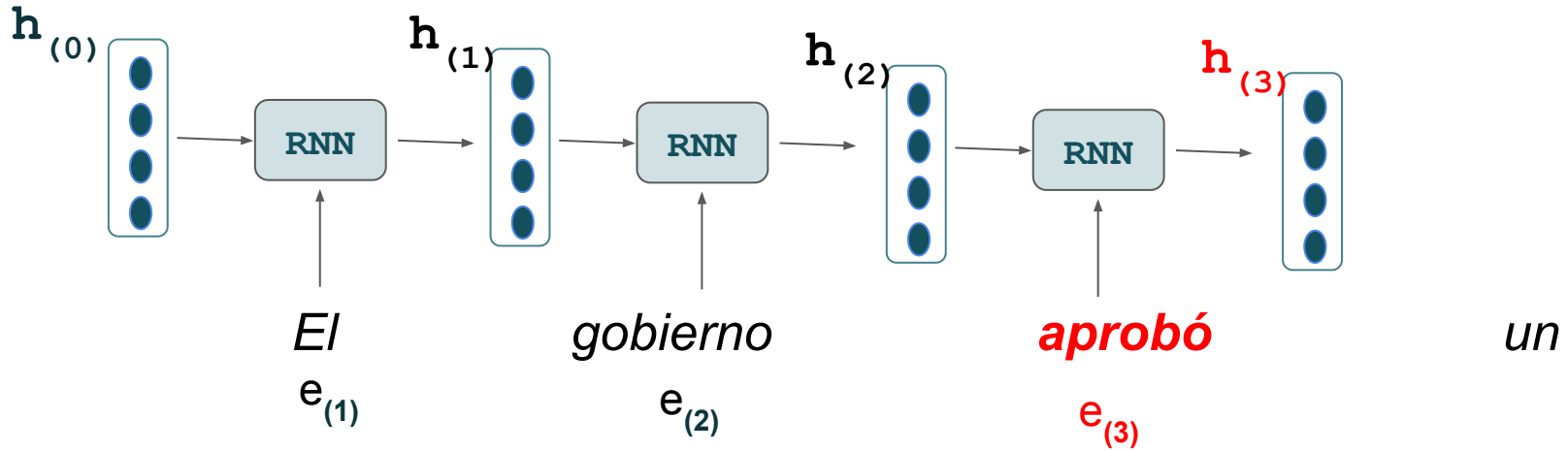
- Como no hay estado anterior, creamos un estado inicial $\mathbf{h}_{(0)}$



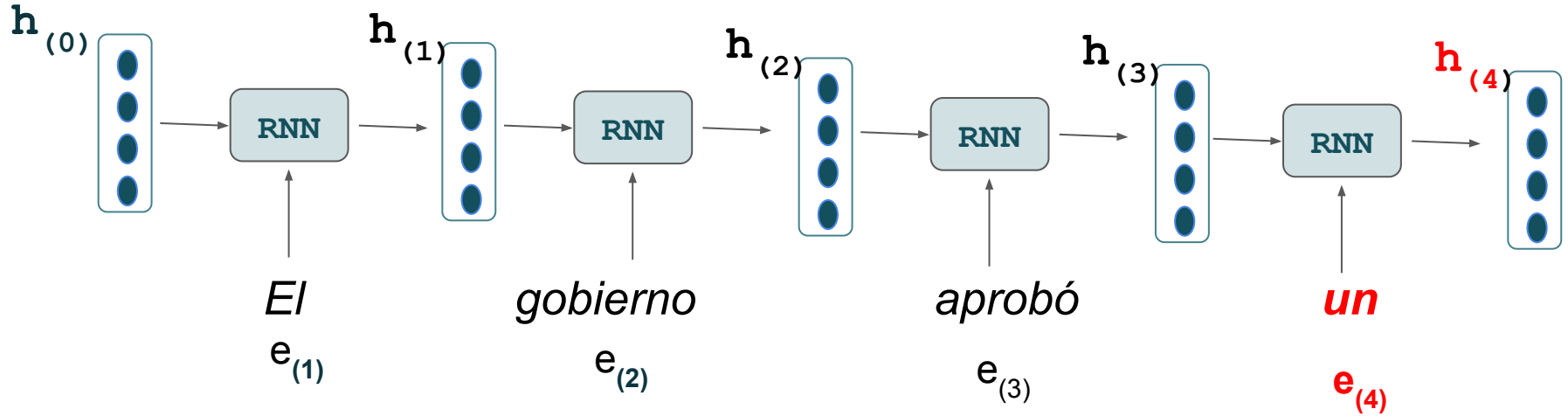
Redes recurrentes (RNN)



Redes recurrentes (RNN)

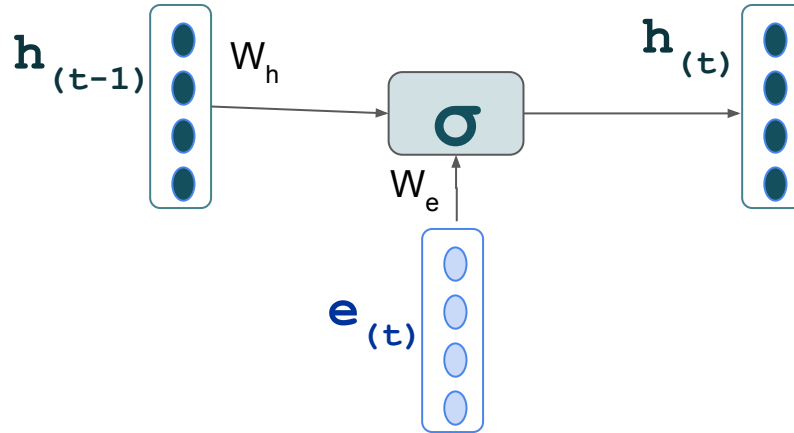


Redes recurrentes (RNN)



Redes recurrentes (RNN) - capa recurrente

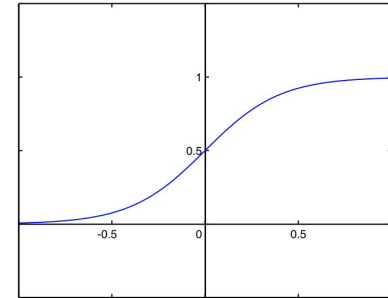
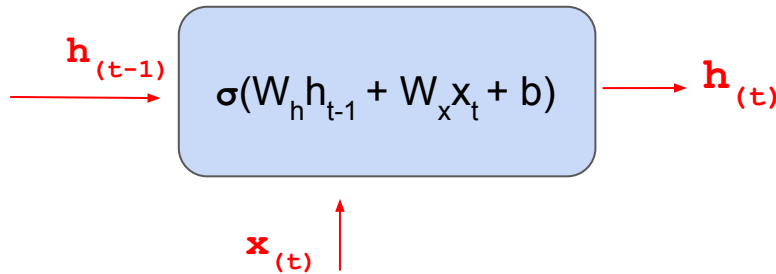
En una **RNN vanilla** (versión más básica), **función de activación: tangente hiperbólica (tanh), o sigmoide.**



$e_{(j)}$ vector dimensión d

Tipos de RNN: básica (vanilla)

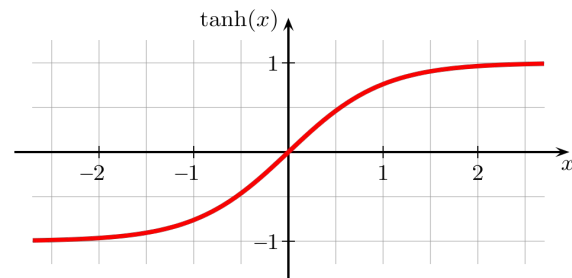
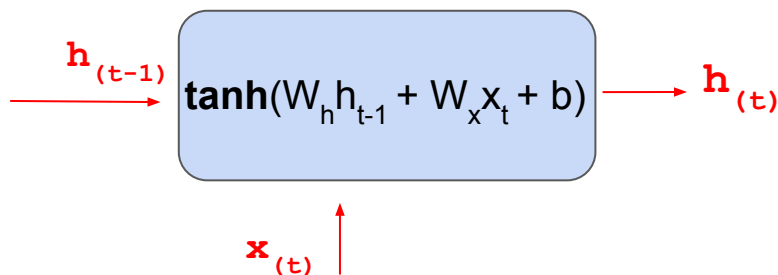
- **sigmoide** garantiza que todo los valores están entre $[0,1]$.



Chrislb, CC BY-SA 3.0
<<http://creativecommons.org/licenses/by-sa/3.0/>>, via
Wikimedia Commons

RNN vanilla: capa recurrente

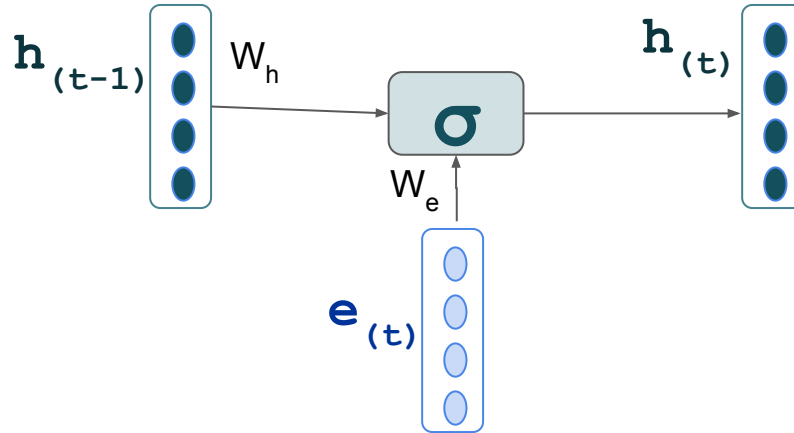
- Del mismo modo, **tanh** también asegura que todos los valores de los vectores siempre estén en $[-1,1]$.



Geek3, CC BY-SA 3.0 <<https://creativecommons.org/licenses/by-sa/3.0/>>, via Wikimedia Commons

Redes recurrentes (RNN) - capa recurrente

En una **RNN vanilla** (versión más básica), **función de activación: tangente hiperbólica (tanh), o sigmoide**.



- acotan los valores
- introducen **no linealidad**
- son **derivables** (necesario para backpropagation)

$e_{(j)}$ vector dimensión d

Redes recurrentes (RNN) - capa recurrente

$$\mathbf{h}_{(t)} = \sigma(W_h \mathbf{h}_{(t-1)} + W_e \mathbf{e}_{(t)} + \mathbf{b})$$

- **$\mathbf{h}(t)$** : estado oculto en el paso temporal t .. Es la *memoria* de la red tras procesar el token $x(t)$.
- **$\mathbf{h}(t-1)$** : estado oculto del paso anterior (información del pasado).
- **$\mathbf{e}(t)$** : entrada en el paso t . En PLN, normalmente es **el embedding del token $x(t)$** .
- **W_h** : matriz de pesos recurrentes (estado $\rightarrow$ estado).
- **W_e** : matriz de pesos de entrada (embedding $\rightarrow$ estado).
- **$\mathbf{b}$** : sesgo

Redes recurrentes (RNN) - capa recurrente

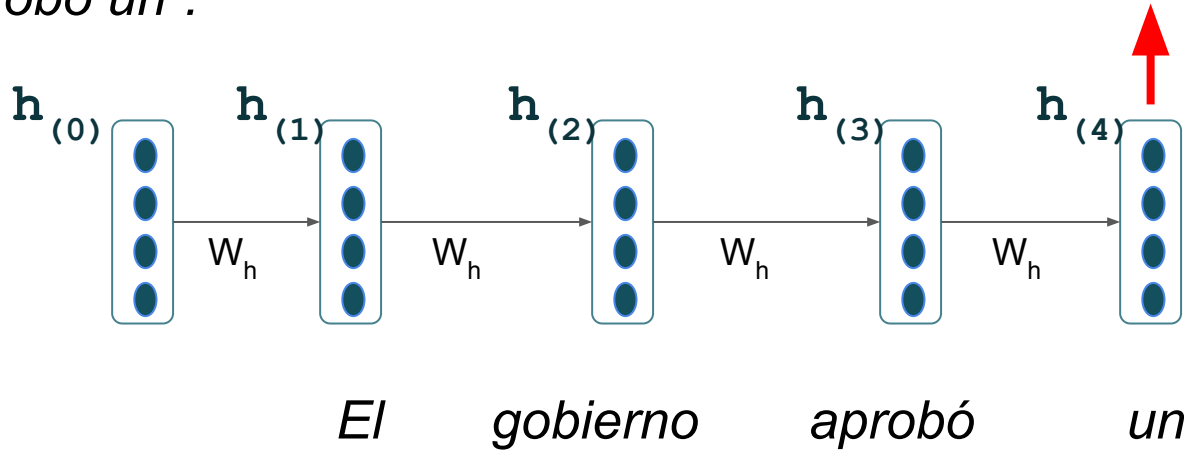
$$\mathbf{h}_{(t)} = \sigma(W_h \mathbf{h}_{(t-1)} + W_e \mathbf{e}_{(t)} + \mathbf{b})$$

RNN básica (ejemplo con $d_e = 300$, $d_h = 128$)

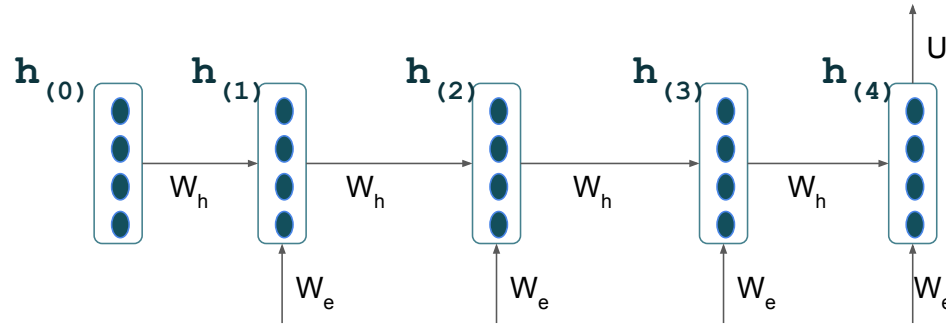
- $e(t) \in \mathbb{R}^{300} \rightarrow$ embedding del token en el paso t
- $h(t-1) \in \mathbb{R}^{128} \rightarrow$ estado oculto anterior
- $h(t) \in \mathbb{R}^{128} \rightarrow$ estado oculto actual
- $W_e \in \mathbb{R}^{128 \times 300}$
- $W_h \in \mathbb{R}^{128 \times 128}$
- $b \in \mathbb{R}^{128}$

Capa de salida

- El último estado es la entrada de la capa de salida.
- En nuestro caso, el **objetivo de la red es predecir el token** que con **mayor probabilidad** continúe la secuencia “*El gobierno aprobó un*”.



Redes recurrentes (RNN) - capa de salida



U también son pesos de la red, que serán ajustados durante el entrenamiento

$$\text{softmax}(U h_{(4)}^t + b_2) \in \mathbb{R}^{|V|}$$

La función debe devolver una probabilidad para cada palabra del vocabulario.

- h_t (estado oculto final) $\in \mathbb{R}^{d_h}$
- U (matriz de pesos de la capa de salida) $\in \mathbb{R}^{|V| \times d_h}$
- b_2 (sesgo de la capa de salida) $\in \mathbb{R}^{|V|}$
- $U h_t$ (logits) $\in \mathbb{R}^{|V|}$
- $U h_t + b_2$ (logits con sesgo) $\in \mathbb{R}^{|V|}$
- $\text{softmax}(U h_t + b_2)$ (probabilidades) $\in \mathbb{R}^{|V|}$

Redes recurrentes (RNN)

- Por ejemplo, dado el siguiente último estado, softmax produce las siguientes probabilidades:

0,1
0,000001
0,05
0,0000001
0,9
0,0000001
0,2
0,0000001
0,0000001



$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

donde $n=|V|$



0,101
0,092
0,0976
0,092
0,226
0,0923
0,112
0,092
0,092
1

La suma de las probabilidades es 1

Redes recurrentes (RNN) - capa de salida

- La **función softmax** devuelve un vector de tamaño $|V|$, donde cada valor se corresponde con la probabilidad de un token en el V.
- Finalmente, la red devuelve el token con mayor probabilidad (en nuestro ejemplo, **decreto**)

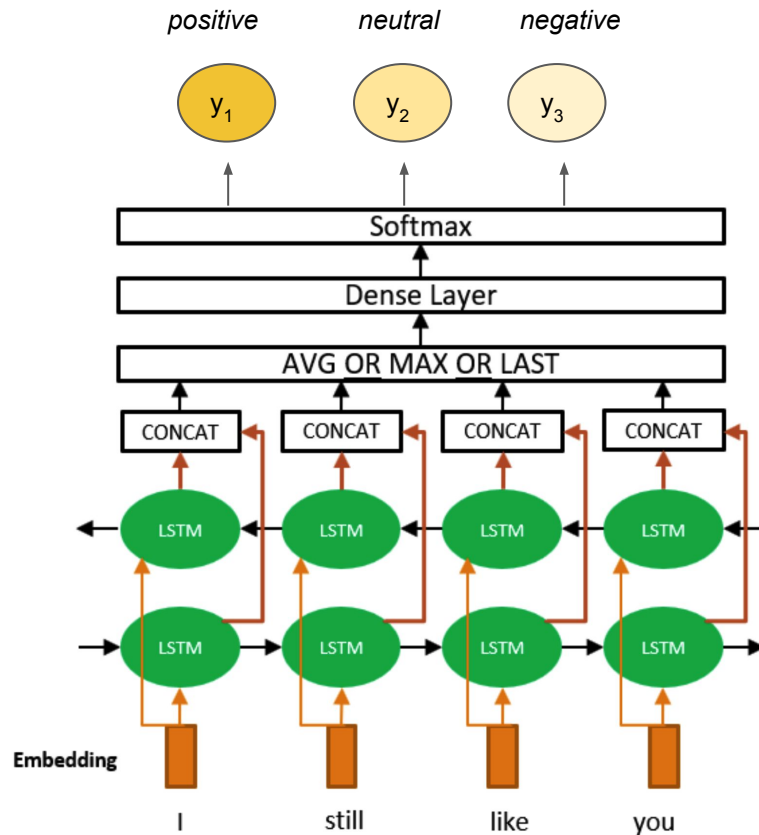


*El gobierno aprobó un **decreto***

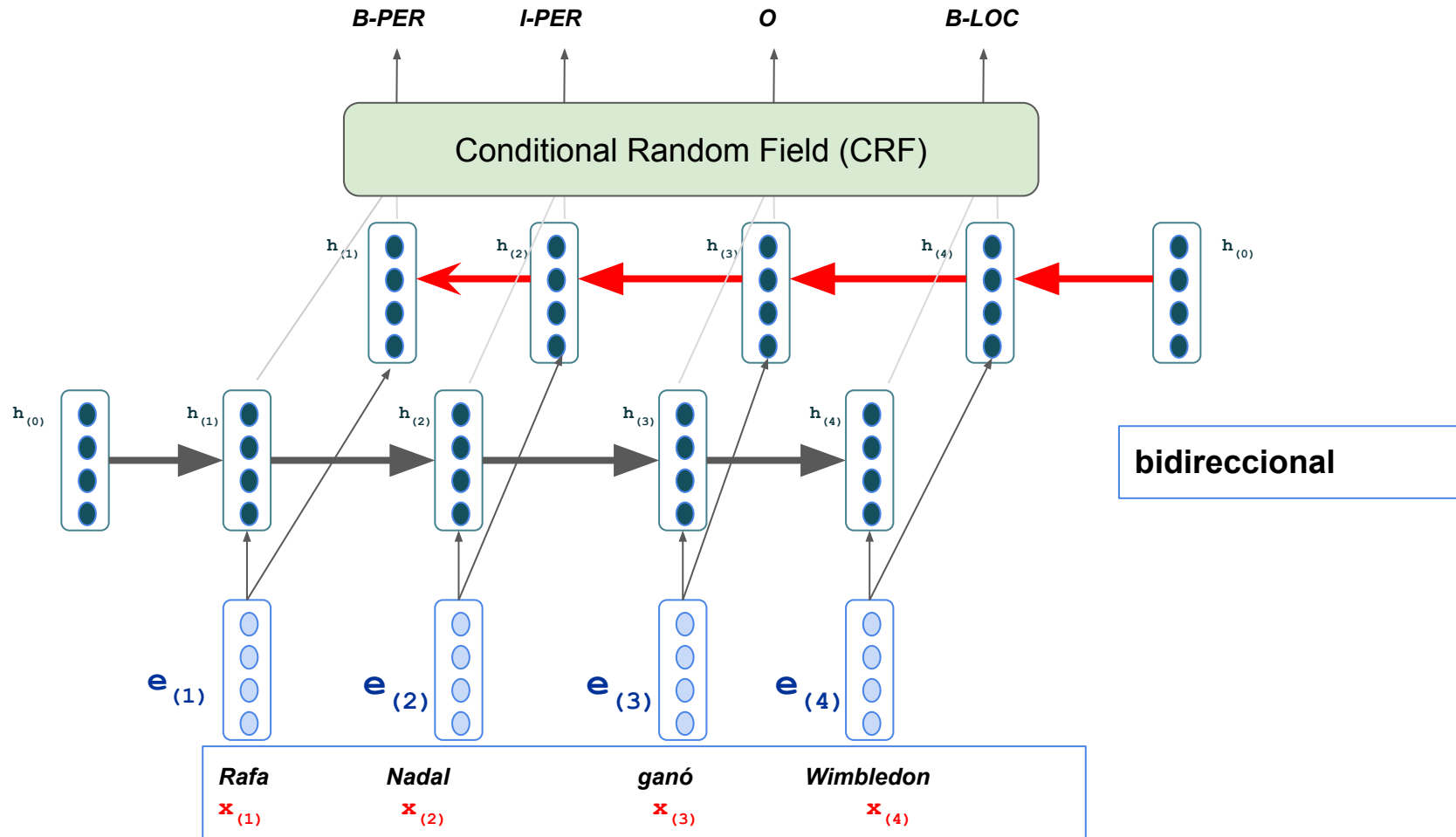
Capa de salida depende del problema

- Modelado del lenguaje (predicción del próximo token): softmax (distribución sobre el vocabulario)
- Clasificación binaria: sigmoide
- Clasificación multiclase: softmax (probabilidades por cada clase)
- Regresión (predice un valor real): no necesita función de activación. Devuelve: $Uh_{(t)} + b$
- Etiquetado secuencial (NER, POS)
 - Conditional Random Field
 - softmax (sobre el conjunto de etiquetas)

Clasificación de textos

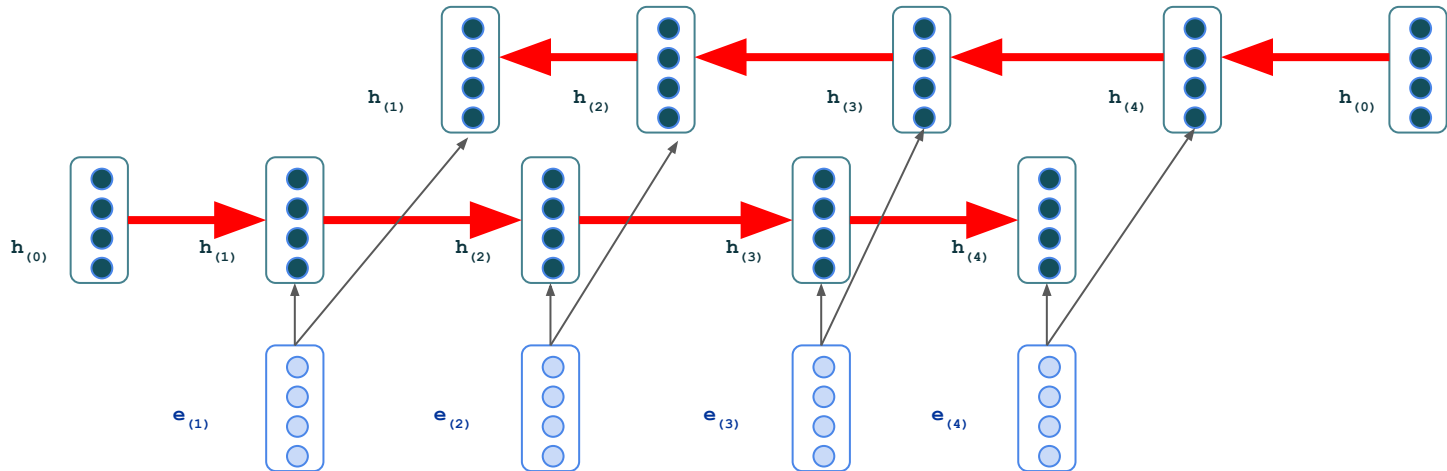


Reconocimiento de Entidades



Redes recurrentes (RNN) - bidireccionales

- En otras tareas distintas a la predicción del siguiente token, siempre es recomendable **añadir** una **capa** que pueda **leer la secuencia de derecha a izquierda** (desde el último token al primero).



Índice

- Redes recurrentes
- **Tipos de RNN**
 - básica (vanilla)
 - LSTM
 - GRU
- Aplicaciones de RNN en PLN

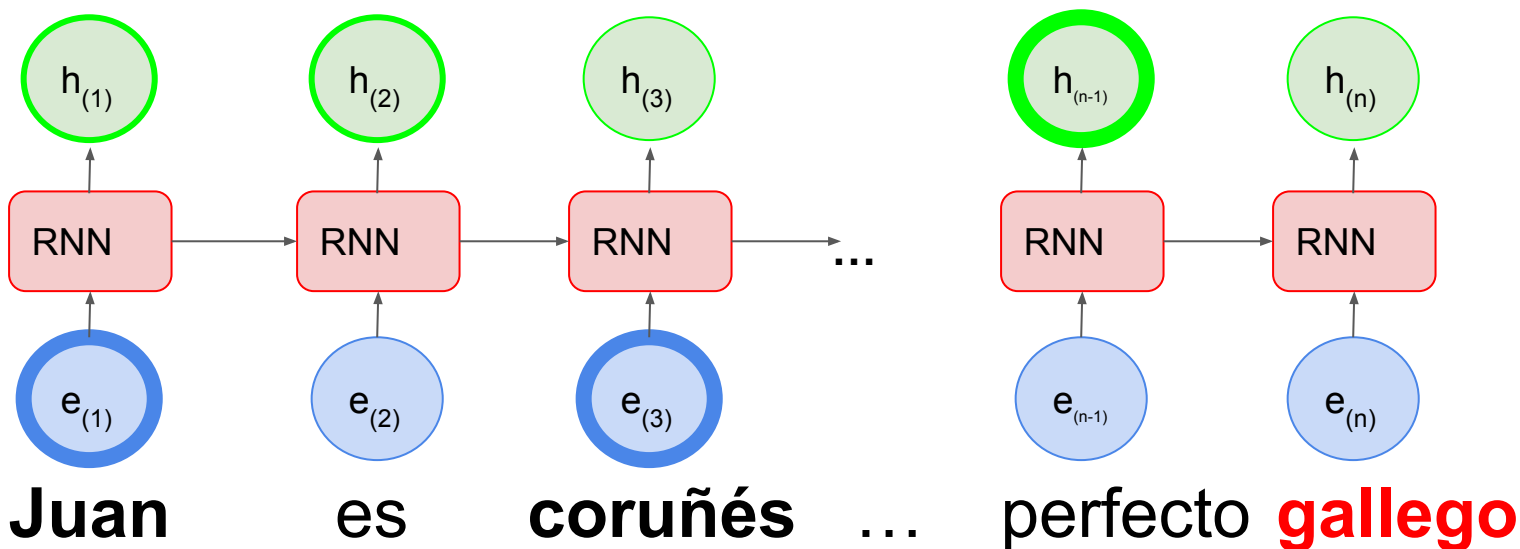
Tipos de RNN

- Clasificación basada en la estructura de la neurona.
 - **básica (vanilla)**: únicamente usa la función **tangente hiperbólica** o **sigmoide**.
 - LSTM: combina distintas funciones de activación para definir tres puertas para controlar el flujo de la información.
 - GRU: también combina distintas funciones de activación aunque únicamente tiene dos puertas.

Limitaciones RNN vanilla

- **Procesar secuencias largas.**
- Desvanecimiento del gradiente

Juan es coruñés. Se crió en un barrio muy humilde, y durante sus primeros años trabajó como chico de los recados, antes de emigrar a Madrid. Él habla un perfecto gallego.



Limitaciones RNN vanilla

- Procesar secuencias largas.
- **Desvanecimiento del gradiente**

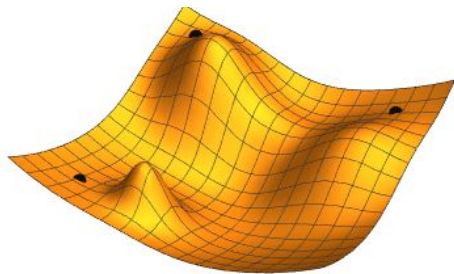
Limitaciones RNN (vanishing gradient)

- El **desvanecimiento del gradiente** ocurre cuando el **gradiente** va tomando **valores muy pequeños**. Esto provoca que los **parámetros no se modifiquen eficazmente**.

https://miro.medium.com/v2/resize:fit:1400/format:webp/1*-rPcuafUSo-SwZF121y_rQ.png

Gradiente descendiente

- Durante la fase de entrenamiento, el **objetivo** es aprender los **parámetros** (pesos) W que **minimizan la función de coste** (loss), es decir, debemos encontrar un mínimo.



Jacopo Bertolotti, CC0, via Wikimedia Commons

- Para ello la red utiliza el método **gradiente descendiente** (*gradient descent*). Es un método de optimización numérica para estimar los mejores parámetros.

Gradiente descendiente

- El **gradiente** es el conjunto de todas las **derivadas parciales** de una función.
- En el método de **gradiente descendiente**, se calculan las derivadas parciales de la **función de coste (loss)**.

W : parámetros de la red

m : número de ejemplos

$$J(W) = \frac{1}{2m} \sum_{i=1}^m [h_W(x_i) - y_i]^2$$

$h_W(x_i)$: predicción para x_i

y_i : valor para x_i en el dataset

Gradiente descendiente

- Una vez calculadas las derivadas parciales, cada parámetro es actualizado con la siguiente ecuación:

α : ratio de aprendizaje (learning ratio)

$$W_i = W_i - \alpha \frac{\partial}{\partial W_i} J(W)$$

- Es decir, cada parámetro es actualizado proporcionalmente a la derivada parcial de la función de coste con respecto al parámetro actual, en cada iteración del entrenamiento.

Gradiente descendiente

α : ratio de aprendizaje (learning ratio)

$$W_i = W_i - \alpha \frac{\partial}{\partial W_i} J(W)$$

- A **mayor gradiente**, **mayor** será el **cambio en W_i** . Por el contrario, a **menor gradiente**, **menor** será el **cambio en W_i** .
- La elección de **ratio de aprendizaje** es muy importante.
 - Si es **demasiado pequeño**, el **método tardará mucho tiempo** en encontrar la solución óptima.
 - Si es **demasiado grande**, los **cambios en W** también serán **muy grandes** y será muy **difícil** encontrar los **parámetros** que **minimicen la función de coste**.
- Ratio de aprendizaje más utilizados: 0.1, 0.01

Desvanecimiento del gradiente en RNN básica

- ¿Por qué se produce el problema del **desvanecimiento del gradiente en las RNN básicas**?
- Estas redes se caracterizan por usar la función de activación **tanh** o **sigmoide**.

- La derivada de la **tanh** es: $\tanh'(x) = 1 - \tanh^2(x)$

- La función **sigmoide** y su derivada son: $\sigma(x) = \frac{1}{1 + e^{-x}}$
 $\sigma'(x) = \frac{e^{-x}}{(1 + e^{-x})^2}$

Desvanecimiento del gradiente en RNN básica

- Los valores de las derivadas de **tanh** siempre están **entre 0 y 1**.
- **Sigmoidea** produce gradientes aún más pequeños (valores **entre 0 y 0.25**).



Desvanecimiento del gradiente en RNN básica

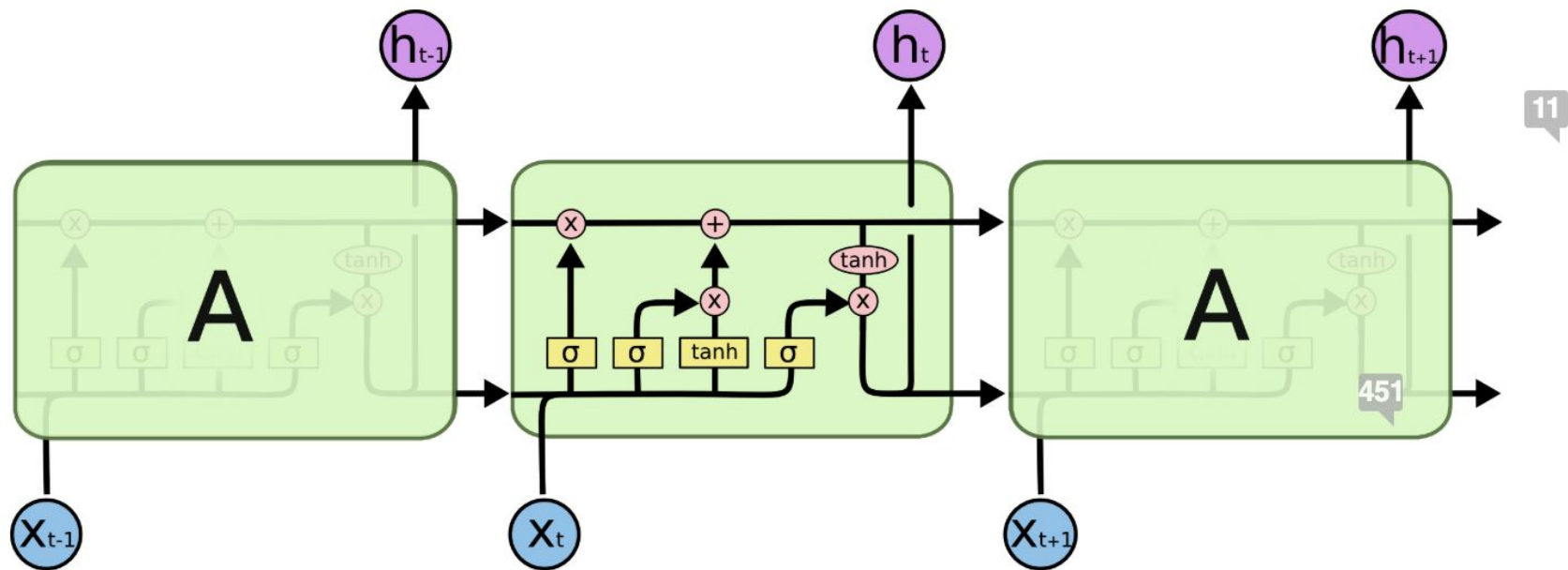
- En las RNN básicas, a **mayor número de capas**, durante el proceso de retropropagación (back-propagation), los **gradientes** serán **cada vez más pequeños**, y por tanto, **no se producirán cambios significativos en los parámetros**.
- Evitar el **problema del desvanecimiento del gradiente** sigue siendo un **área de investigación activa**.
- Posibles soluciones:
 - **LSTM** (Long-short Term Memory Networks)*, también sufre el mismo problema, pero más lentamente que en RNN básicas.

*Hochreiter, S., Bengio, Y., Frasconi, P., & Schmidhuber, J. (2001). Gradient flow in recurrent nets: the difficulty of learning long-term dependencies.

Índice

- Redes recurrentes
- Otros ejemplos de aplicaciones de RNN en PLN
- Tipos de RNN:
 - básica (vanilla)
 - **LSTM (Long-short Term Memory Networks)**
 - GRU

Tipos de RNN: LSTM



- [tutorial](#)

Ventajas de LSTM

- **Detectar las dependencias en secuencias largas,**
- Disminuyen el **problema del desvanecimiento del gradiente.**
- Son más robustas (manejar el ruido o la falta de datos) en comparación a otras redes.
- Pueden ser utilizadas para una gran variedad de aplicaciones: clasificación de textos, generación de textos, reconocimiento de entidades, etc.

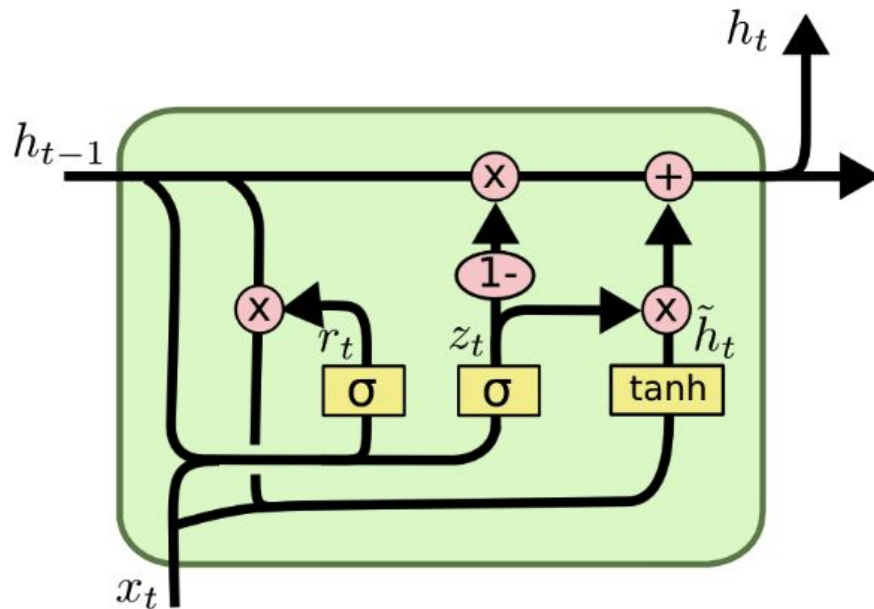
Desventajas de LSTM

- Complejidad temporal es alta.
- Gran número de hiper-parámetros.
- Necesitan ser entrenados con datasets grandes.
- Tienden al sobreaprendizaje en datasets pequeños.

Índice

- Redes recurrentes
- Otros ejemplos de aplicaciones de RNN en PLN
- Tipos de RNN:
 - básica (vanilla)
 - LSTM (Long-short Term Memory Networks)
 - **GRU**

GRU - Gated recurrent units*



[Tutorial](#)

Ventajas GRU

- Como son más sencillas que las LSTM, son **más eficientes**.
- **Mejor elección** que las LSTM si la **arquitectura** es **sencilla** o se cuentan con **pocos recursos** computacionales.

Desventajas GRU

- Sin embargo, muestran **peores resultados** en el tratamiento de **secuencias largas**.
- También presentan más problemas de **sobreaprendizaje** que las LSTM, en especial cuando se trabaja con datasets pequeños.
- Al igual que las LSTM, para obtener **buenos resultados** es necesario **ajustar** sus hiper-**parámetros** (número de unidades, learning rate,

Índice

- Redes recurrentes
- Tipos de RNN
 - básica (vanilla)
 - LSTM
 - GRU
- **Aplicaciones de RNN en PLN**

Implementación

- Aquí tienes una implementación de un enfoque para clasificación binaria basada en [BiLSTM](#)
- Ejercicio:
 - Prueba otras configuraciones de la BiLSTM, ¿consigue mejorar los resultados?
 - Crea una implementación para clasificación multi-clase en el dataset EXIST 2021.

Resumen

- **RNN** son apropiadas para **procesar secuencias**.
- Toman **decisiones** en función del **pasado** (estados anteriores).
- Su principal **desventaja** es que **no** puede ser **paralelizada**.
- Problemas de una **RNN básica** (tanh o sigmodea): **no detectan dependencias en secuencias largas y desvanecimiento del gradiente**.
- **LSTM**, tipo de RNN, pueden **capturar dependencias entre elementos distantes**.
- **GRU** más simple y **eficiente** que LSTM.

OpenCourseWare
Procesamiento de Lenguaje Natural con
Aprendizaje Profundo,

Gracias!!!

<https://github.com/iseaura>